Student Tracking System

Project Practicum

Submitted By

Rathanak Phan

Rourn Soaly

Phen Noreak

Nov SingJu

Supervisor

TOUCH NGUONCHHAY

Faculty of Engineering

Royal University of Phnom Penh

Submit Date

Acknowledgment

We would like to express our sincere gratitude to our supervisor, Mr. Touch Nguonchhay, for his valuable guidance, constructive feedback, and continuous support throughout the development of this project. His academic insight and practical advice played a crucial role in shaping the quality of this system.

We also thank the Faculty of Engineering, Royal University of Phnom Penh, for providing the academic foundation, technical resources, and learning environment necessary for completing this practicum.

Finally, we express our heartfelt appreciation to our families and friends for their encouragement, motivation, and support during the project period.

Table of Contents

List of Figures

1. Introduction

The rapid development of information technology has significantly improved how educational institutions manage and process student data. This project, titled **Student Tracking System**, is developed as part of the **Project Practicum** course at the Faculty of Engineering, Royal University of Phnom Penh.

The main goal of this project is to design and implement a web-based system that helps administrators, teachers, and users manage student information efficiently and securely.

1.1 Problem Description and Statement

Many educational institutions still rely on manual or semi-digital methods such as paper records or spreadsheets to manage student data. These methods often lead to data inconsistency, duplication, data loss, and difficulty in tracking student information efficiently.

As the number of students increases, managing records manually becomes more complex and time-consuming. Therefore, a centralized and automated student tracking system is required to improve accuracy, accessibility, and efficiency.

1.2 Objectives

The objectives of this project are:

- To design and develop a web-based Student Tracking System
- To efficiently and securely manage student data
- To implement role-based access for Admin, Teacher, and Student users
- To improve data accuracy and reduce manual administrative work
- To provide a responsive and user-friendly interface

2. Project Review

2.1 Existing Project 1 Review

“Fedena”

Fedena is a comprehensive **Student Information System (SIS)** used by schools and universities worldwide. It provides features for managing student profiles, enrollment, attendance, grades, academic history, and user roles.

The system supports **role-based access control**, allowing administrators, teachers, and students to access different system functionalities. Administrators manage users and system settings, teachers manage classes and assessments, and students can view their academic records.

Fedena is very similar to the proposed Student Tracking System in terms of **centralized student data management** and **role separation**.

Key Features:

- Student profile and academic history tracking
- Admin, Teacher, and Student roles
- Attendance and grade management

- Centralized database

Limitations:

- Commercial system (paid license)
- Customization requires additional cost
- UI may be complex for small institutions

🔗 Official Website: https://fedena.com

2.2 Existing Project 2 Review

“openSIS”

openSIS is an **open-source Student Information System** designed for schools and higher education institutions. It allows administrators to manage student records, teacher assignments, courses, grades, and attendance through a web-based interface.

openSIS closely matches the proposed system because it focuses on **tracking student data rather than online teaching**. It supports multiple user roles and provides a structured database for academic records.

Key Features:

- Student registration and profile management
- Role-based access (Admin, Teacher, Student)
- Academic records and grading
- Web-based system

Limitations:

- Outdated UI/UX compared to modern systems
- Limited scalability without paid version
- Requires technical knowledge for customization

🔗 Official Website:
https://opensis.com

2.3 Existing Project 3 Review

“PowerSchool”

PowerSchool is one of the most widely used **Student Information Systems (SIS)** in educational institutions. It provides advanced tools for managing student data, attendance, grades, transcripts, and administrative workflows.

PowerSchool is designed specifically for **student tracking and academic administration**, making it highly relevant to the proposed system. It includes dashboards for administrators, teachers, and students, along with reporting and analytics tools.

Key Features:

- Centralized student data management
- Detailed academic performance tracking
- Admin, Teacher, and Student dashboards
- Reporting and analytics

Limitations:

- Proprietary and expensive
- Closed-source
- Overkill for small institutions

🔗 Official Website:
https://www.powerschool.com

2.4 Comparison with Proposed Student Tracking System

Unlike the systems above, the proposed **Student Tracking System** is designed as a **lightweight, customizable, and educational-purpose-focused solution** tailored for academic institutions such as the Royal University of Phnom Penh. It emphasizes:

- Simplicity and usability
- Full control over data and system logic
- Custom role-based workflows (Admin, Teacher, Student)
- Cost-effective deployment using open-source technologies

3. Methodology

3.1 Agile Approach Overview

The **Student Tracking System** was developed using a **Practical Agile Software Development Methodology**. Agile methodology was chosen for this project because it supports **iterative development**, **continuous feedback**, and **flexibility** in responding to changing requirements during the development process.

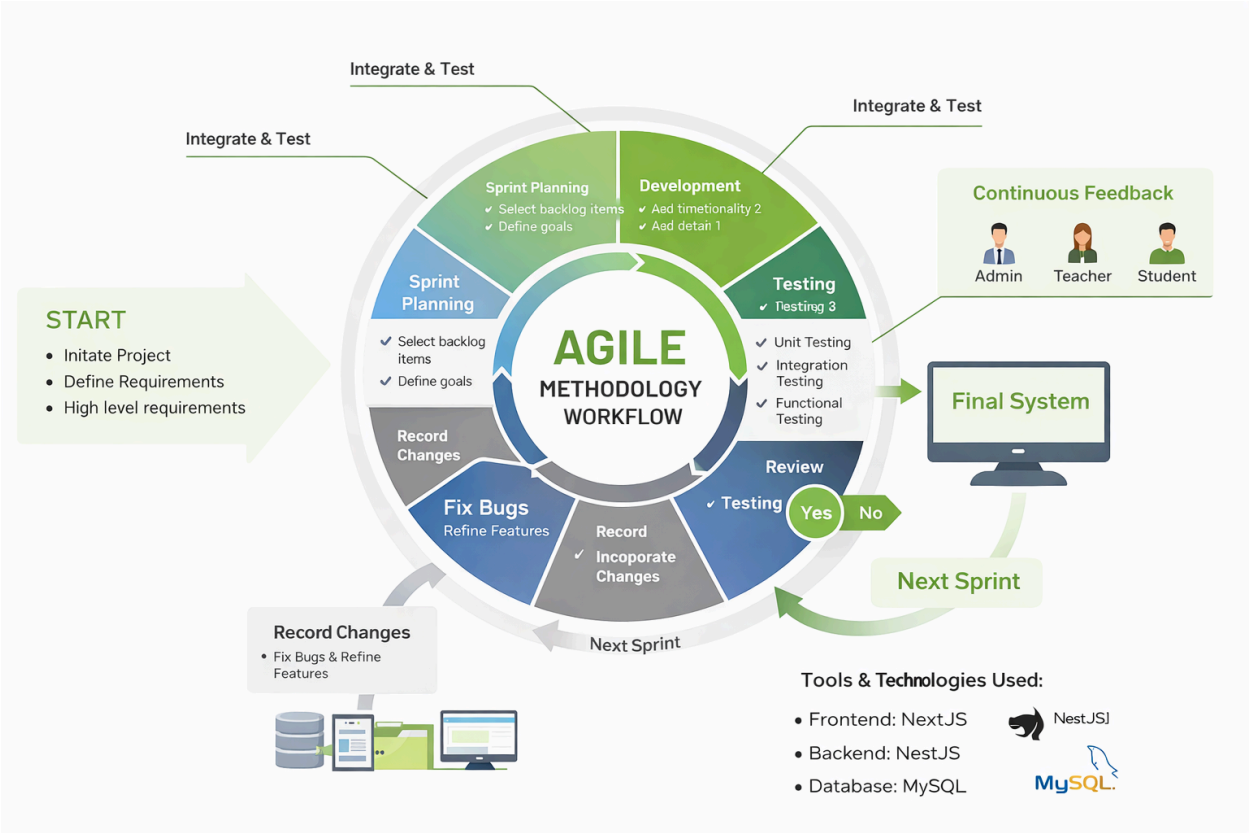
Unlike traditional linear approaches, Agile allows the system to be built incrementally through multiple development cycles, known as **sprints**. Each sprint delivers a functional and testable part of the system, enabling early validation, continuous improvement, and reduced development risk. This approach is well suited for a web-based academic system involving multiple user roles such as administrators, teachers, and students.

3.2 Requirements Gathering

The Agile workflow applied in this project follows a **cyclical and iterative process**, as illustrated in **Figure 3.1**. The workflow begins with project initiation and continues

through sprint planning, development, testing, review, and refinement. Feedback is continuously incorporated to improve system quality and functionality.

Figure 3.1 Agile Methodology Workflow for the Student Tracking System



This diagram represents the Agile development process used in the project, showing how features are developed, tested, reviewed, and improved repeatedly across multiple sprints until the final system is completed.

3.3 Agile Roles and Responsibilities

Agile roles were adapted to fit the academic project environment:

- **Product Owner:**
  The project supervisor, responsible for defining system requirements, setting priorities, and providing feedback during sprint reviews.

- **Scrum Master:**
  The team leader, responsible for coordinating sprint activities, managing progress, and ensuring Agile practices were followed.

- **Development Team:**
  Student developers, responsible for system design, frontend and backend implementation, testing, and documentation.

This role structure ensured clear responsibility and effective collaboration throughout the project lifecycle.

3.4 Agile Development Process

Each sprint followed a structured Agile cycle consisting of the following stages:

3.4.1 Project Initiation and Requirement Definition

At the beginning of the project, high-level requirements were identified, including system scope, user roles, and core functionalities such as student management, class enrollment, and academic tracking.

3.4.2 Sprint Planning

Sprint planning was conducted at the start of each sprint to:

- Select prioritized features from the product backlog
- Define sprint objectives and tasks
- Allocate responsibilities among team members

3.4.3 Design

During the design phase:

- Database structures were designed using MySQL
- Use case diagrams were created to model user interactions
- UI/UX designs were developed using Figma to ensure usability and responsiveness

3.4.4 Development

System development was carried out using:

- Frontend: NextJS, Tailwind CSS, TypeScript
- Backend: Node.js + Express (RESTful API architecture)

Features were implemented incrementally, allowing early testing and validation.

3.4.5 Integration and Testing

After development, features were integrated and tested using:

- Unit testing
- Integration testing
- Functional testing

Testing ensured that system components worked correctly together and met defined requirements.

3.4.6 Review and Feedback

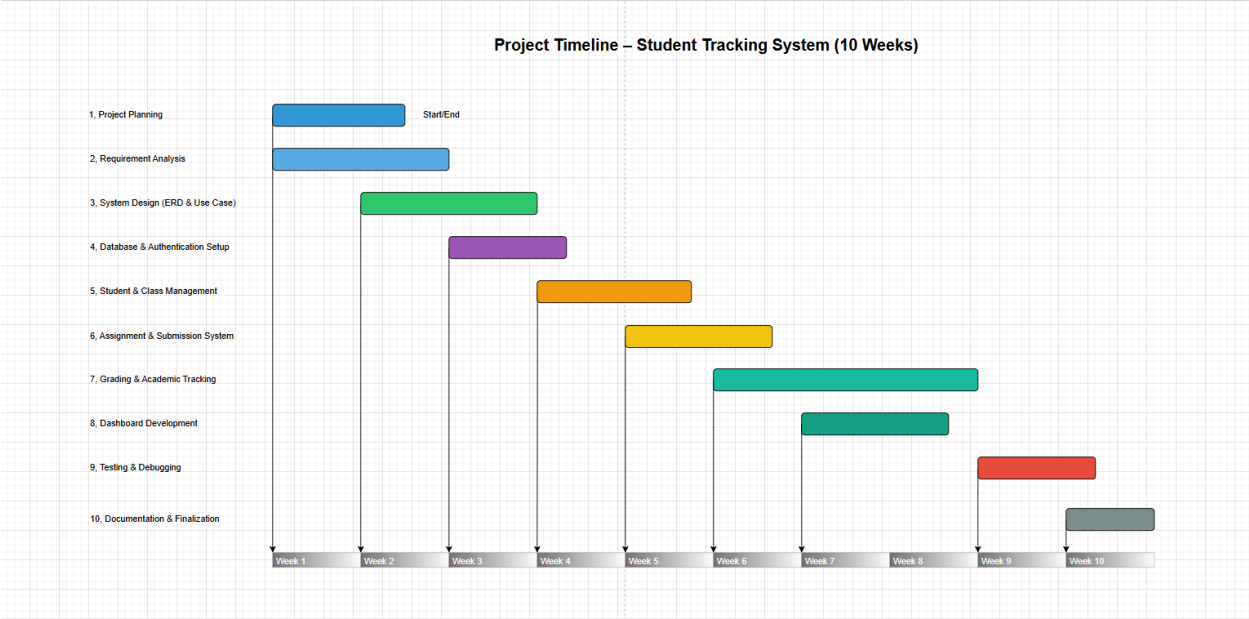
At the end of each sprint, completed features were reviewed. Feedback from the supervisor and users was collected and used to improve system functionality, performance, and usability.

3.4.7 Refinement and Next Iteration

Identified issues and improvement suggestions were recorded and incorporated into the next sprint. This iterative process continued until all planned features were completed.

3.5 Agile Project Timeline

The development of the Student Tracking System was completed over a period of ten weeks using an Agile-inspired development methodology. The timeline was divided into several development phases, beginning with project planning and requirement analysis, followed by system design, implementation of core functionalities, and finally testing and documentation. Each stage contributed to building a fully functional and reliable system.



N	Timeline	Task	Main activities	Description
1	Week	Project		During this phase, the

	1	Planning	- Define project objectives - Identify system scope - Determine system users (Admin, Teacher, Student) - Prepare initial project proposal	development team defined the overall objectives and scope of the Student Tracking System. Discussions with the project supervisor were conducted to determine the main system features and identify key stakeholders. The initial project proposal and development plan were prepared to guide the development process.
2	Week 1–2	Requirement Analysis	- Gather system requirements - Analyze current student data management process - Define functional and non-functional requirements - Document system requirements	In this phase, the development team analyzed the system requirements and identified the functionalities needed for the Student Tracking System. The team studied existing student management methods and defined features such as student profile management, class enrollment, assignment submission, and academic tracking.
3	Week 2–4	System Design (ERD &	- Design system architecture - Create Use Case	During this phase, the overall system structure and database design were

		Use Case)	Diagram - Develop Entity Relationship Diagram (ERD) - Define database entities and relationships	developed. The Use Case Diagram was created to illustrate how users interact with the system, while the ERD defined the relationships between entities such as users, classes, assignments, and submissions.
4	Week 3–4	Database & Authentication Setup	- Implement MySQL database schema - Create database tables - Develop login and registration system - Implement role-based access control	In this stage, the database structure was implemented based on the ERD design. Authentication features such as login and registration were developed to allow users to securely access the system. Role-based access control ensured that each user could only access the appropriate system features.
5	Week 4–6	Student & Class Management	- Implement student profile management - Develop class creation feature - Enable student enrollment in classes - Manage class information	This phase focused on implementing the core academic management features of the system. Administrators were able to manage student accounts, teachers could create classes, and students could join classes and access learning materials.

6	Week 5–6	Assignment & Submission System	- Create assignment management system - Implement assignment submission feature - Store submission data in database - Allow teachers to manage assignments	In this phase, the assignment system was implemented. Teachers were able to create assignments and set deadlines, while students could submit assignments through the system. The submission data was stored and managed within the database.
7	Week 6–8	Grading & Academic Tracking	- Implement grading functionality - Record student grades - Provide feedback for assignments - Track academic performance	During this stage, the grading system was developed to allow teachers to evaluate student submissions. Academic tracking features were also implemented to monitor student performance and provide insights into learning progress.
8	Week 7–8	Dashboard Development	- Design dashboard interface - Display student progress data - Show assignment and class statistics - Implement user dashboards	In this phase, dashboards were developed to provide users with an overview of system activities. Separate dashboards were created for administrators, teachers, and students to display relevant academic information and

				system statistics.
9	Week 9	Testing & Debugging	- Perform functional testing - Conduct integration testing - Identify system bugs - Fix system errors	This stage focused on ensuring system reliability and performance. Functional and integration testing were conducted to verify that all system components worked correctly. Identified bugs and system issues were resolved to improve system stability.
10	Week 10	Documentation & Finalization	- Finalize project report - Prepare system documentation - Conduct final system review - Prepare system demonstration	In the final phase, the development team completed the project documentation and prepared the system for presentation. The final report was finalized, and the system was reviewed to ensure it met all project requirements before demonstration.

3.6 Tools and Technologies Used

The following tools and technologies were used throughout the development process:

- Frontend Development: NextJS, Tailwind CSS, TypeScript
- Backend Development: Node.js + Express.js
- Database Management: MySQL
- UI/UX Design: Figma
- Diagram Design: Draw.io
- Version Control: GitHub
- Deployment Environment: Docker

These tools enabled efficient development, collaboration, and consistent deployment across environments.

3.7 Testing and Quality Assurance

Testing was integrated into every sprint to ensure system reliability and correctness. The testing activities included:

- **Unit Testing:** Verification of individual modules and functions
- **Integration Testing:** Validation of interactions between system components
- **Functional Testing:** Confirmation that features meet specified requirements
- **Usability Testing:** Evaluation of interface clarity and ease of navigation

All identified defects were resolved before proceeding to subsequent sprints.

3.8 Summary

By applying a Practical Agile Software Development Methodology, the Student Tracking System was developed in an incremental, flexible, and structured manner. The Agile approach enabled continuous testing, regular feedback, and ongoing improvement, resulting in a reliable, user-friendly, and scalable system suitable for academic institutions.

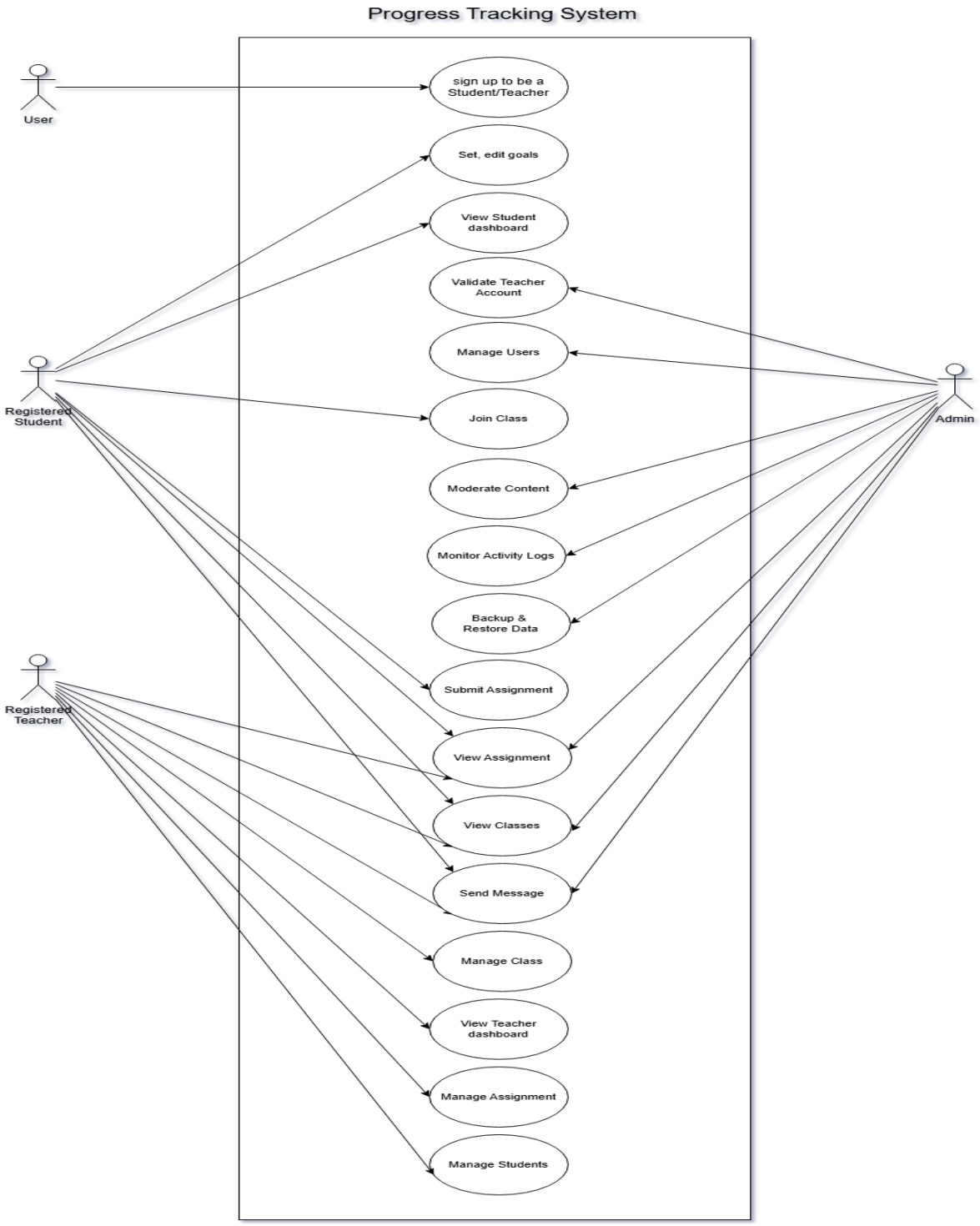
4. Project Design

The project design outlines the overall structure, workflow, and user experience of the student tracking system. It defines how different components—such as data input, storage, processing, and reporting—interact to deliver a seamless solution. The design emphasizes clarity, scalability, and accessibility, ensuring that both administrators and students can navigate the system with ease.

Key aspects include:

- **Use Case Diagram**: Illustrates the interactions between different actors (students, teachers, administrators, and unregistered users) and the system.
- **Database Design**: Defines a simplified, scalable schema to store student information, course data, attendance, and performance records.
- **UI/UX design**: Focuses on intuitive dashboards and user flows that guide users through tasks with minimal friction.

4.1 Use Case Diagram



Actor connections

User --> Sign up/Sign in as Student or Teacher

RegisteredStudent --> SetEditGoals

RegisteredStudent --> ViewStudentDashboard

RegisteredStudent --> ViewAssignment

RegisteredStudent --> SubmiAssignmentt

RegisteredStudent --> JoinClass

RegisteredTeacher --> SubmitAssignment

RegisteredTeacher --> ViewAssignment

RegisteredTeacher --> ViewClasses

RegisteredTeacher --> SendMessage

RegisteredTeacher --> ManageClass

RegisteredTeacher --> ViewTeacherDashboard

RegisteredTeacher --> ManageAssignment

RegisteredTeacher --> ManageStudents

Admin --> ValidateTeacherAccount

Admin --> ManageUsers

Admin --> ModerateContent

Admin --> MonitorActivityLogs

Admin --> SendMessage

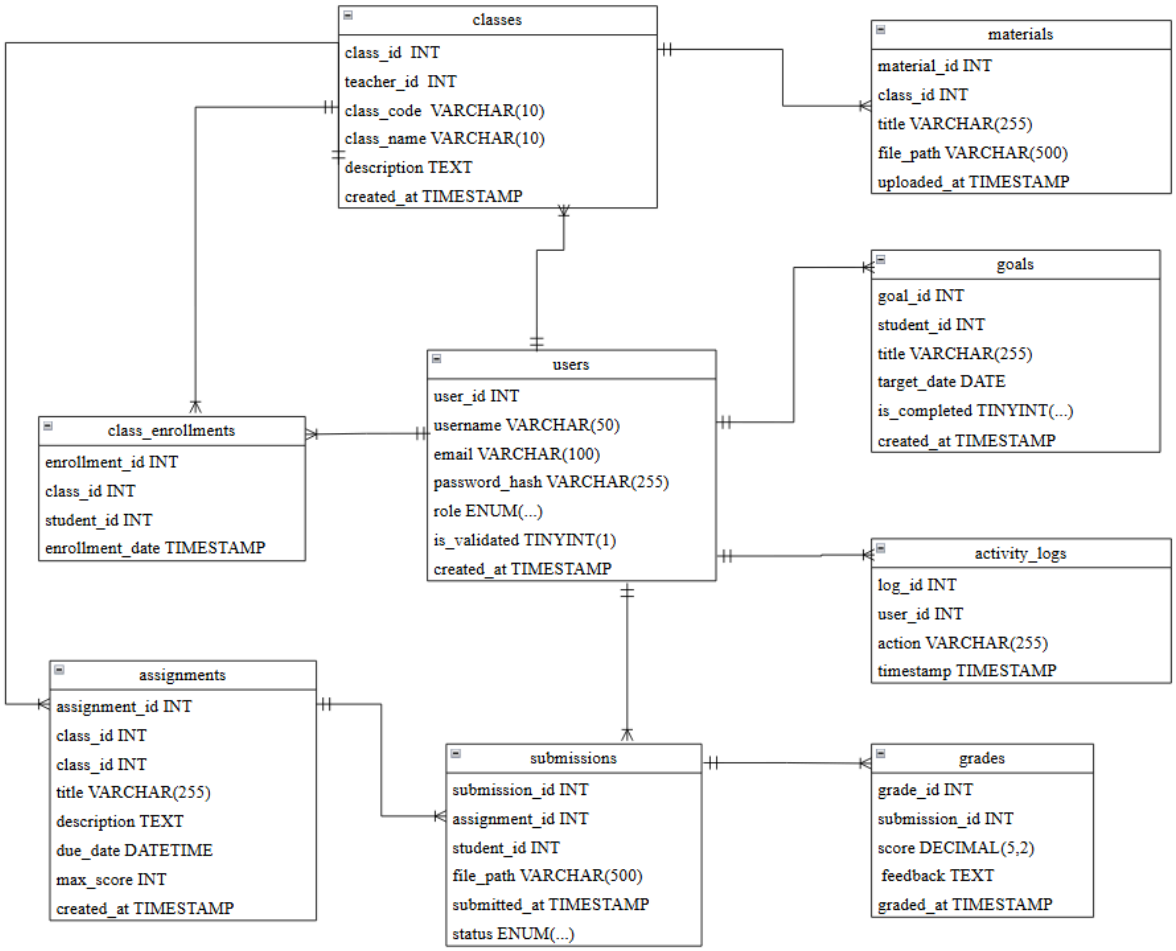
Admin --> ManageClass

Admin --> ManageAssignment

Admin --> BackupRestore

The diagram illustrates the use cases for a Progress Tracking System. It features three actors: User, Registered Student, and Registered Teacher. The User can only sign up to be a Student/Teacher. The Registered Student can set/edit goals, view their student dashboard, and join a class. The Registered Teacher can submit assignments, view assignments, view classes, send messages, manage classes, view their teacher dashboard, manage assignments, and manage students. The Admin has the highest level of access, able to validate teacher accounts, manage users, moderate content, monitor activity logs, and backup/restore data.

4.2 Database Design



Relationships

- Grades → Classes, Assignments, Submissions: grades are associated with classes, specific assignments, and submission records.
- Classes → Materials, Assignments, Goals, Class Enrollments: a class contains learning materials, assignments, goals, and enrollments.
- Assignments → Users, Goals, Submissions: assignments are created/owned by users, linked to goals, and produce submissions.
- Users → Goals, Submissions, Activity Logs: users set goals, make submissions, and generate activity logs.

- Goals → Class Enrollments, Submissions: goals can be tied to enrollments and to submissions that demonstrate progress.
- Class Enrollments → Submissions: enrollments produce submissions for class assignments.
- Activity Logs → Users: activity logs reference the user who performed the action.

Schema Overview

grades

- grade_id INT
- submission_id INT
- score DECIMAL(5,2)
- feedback TEXT
- graded_at TIMESTAMP

classes

- class_id INT
- teacher_id INT
- class_name VARCHAR(10)
- class_code VARCHAR(10)
- description TEXT
- created_at TIMESTAMP

materials

- material_id INT
- class_id INT
- title VARCHAR(255)
- file_path VARCHAR(500)
- uploaded_at TIMESTAMP

assignments

- assignment_id INT
- class_id INT
- title VARCHAR(255)
- description TEXT
- due_date DATETIME
- max_score INT
- created_at TIMESTAMP

users

- user_id INT
- username VARCHAR(50)
- email VARCHAR(100)
- password_hash VARCHAR(255)
- role ENUM(...)
- is_validated TINYINT(1)
- created_at TIMESTAMP

goals

- goal_id INT
- student_id INT
- title VARCHAR(255)
- target_date DATE
- is_completed TINYINT(...)
- created_at TIMESTAMP

class_enrollments

- enrollment_id INT
- class_id INT
- student_id INT
- enrollment_date TIMESTAMP

submissions

- submission_id INT
- assignment_id INT
- student_id INT
- file_path VARCHAR(500)
- submitted_at TIMESTAMP
- status ENUM(...)

activity_logs

- log_id INT
- user_id INT
- action VARCHAR(255)
- timestamp TIMESTAMP

Database schema diagram showing tables: grades, classes, materials, assignments, users, goals, class_enrollments, and activity_logs with their fields and relationships.

4.3 UI/UX design(User Journey)

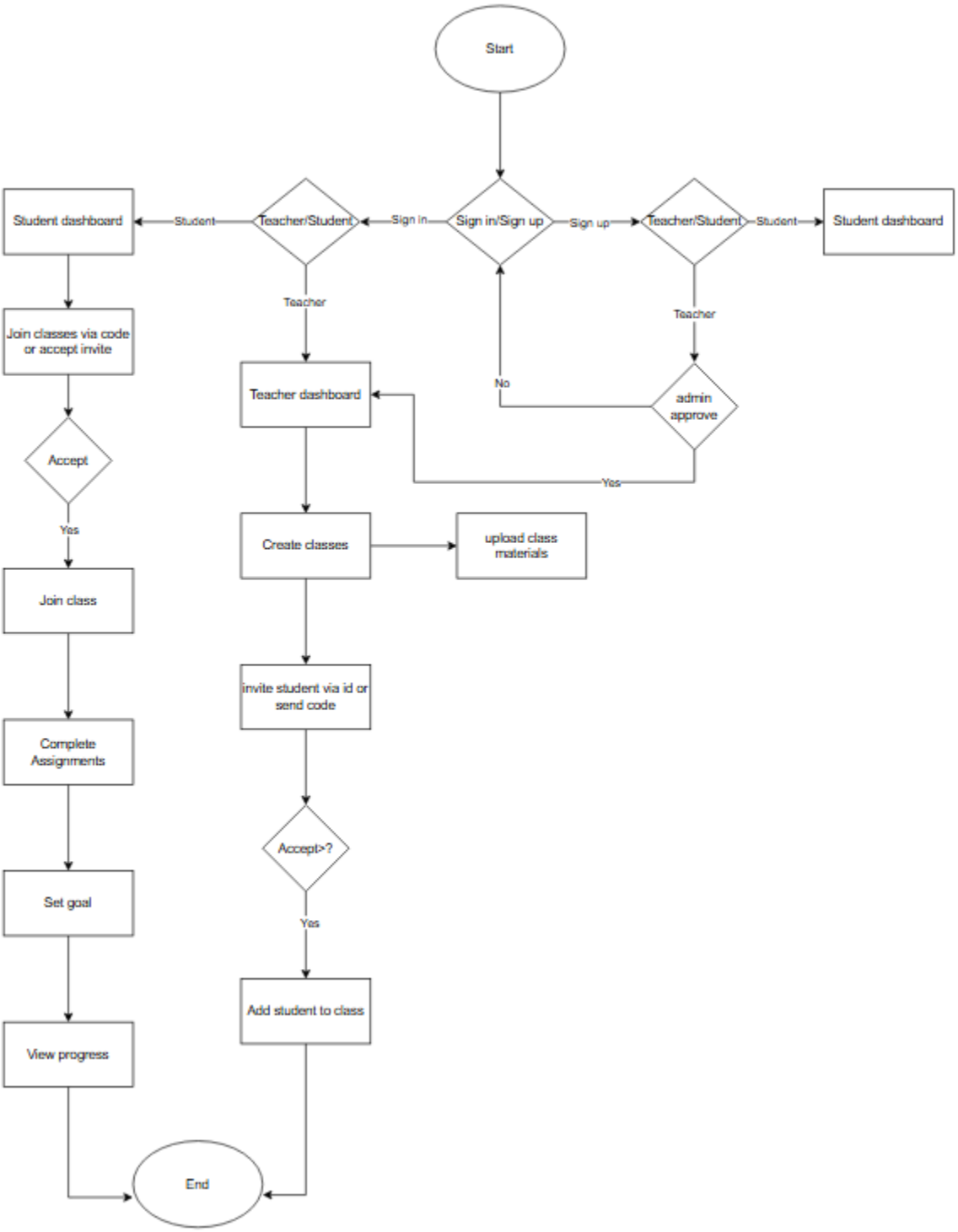
home page

Student dashboard

Teacher dashboard

Admin dashboard

User Flow



Step-by-step Flow

```
Start([Start]) --> SignInSignUp{Sign in or Sign up}

SignInSignUp -- Sign in --> RoleChoice1{Teacher or Student}

SignInSignUp -- Sign up --> RoleChoice2{Teacher or Student}

RoleChoice1 -- Student --> StudentDashboard1[Student dashboard]

RoleChoice1 -- Teacher --> TeacherDashboard[Teacher dashboard]

RoleChoice2 -- Student --> StudentDashboard2[Student dashboard]

RoleChoice2 -- Teacher --> AdminApprove{Admin approval required}

AdminApprove -- No --> SignInSignUp

AdminApprove -- Yes --> TeacherDashboard

TeacherDashboard --> CreateClasses[Create classes]

CreateClasses --> UploadMaterials[Upload class materials]

UploadMaterials --> CreateClasses

CreateClasses --> InviteStudent[Invite student via ID or code]

InviteStudent --> AcceptInvite{Accept invite?}

AcceptInvite -- Yes --> AddStudent[Add student to class]
```

```
AddStudent --> End([End])


StudentDashboard1 --> JoinClasses[Join classes via code or accept invite]

JoinClasses --> AcceptJoin{Accept?}

AcceptJoin -- Yes --> JoinClass[Join class]

JoinClass --> CompleteAssignments[Complete assignments]

CompleteAssignments --> SetGoal[Set goal]

SetGoal --> ViewProgress[View progress]

ViewProgress --> End([End])
```

5 Implementation

5.1 Frontend

The frontend was developed using **Next.js** with **TypeScript** for type-safe, component-based UI development. **Tailwind CSS** was used for responsive and consistent styling across all pages. The application supports three distinct dashboard views — Admin, Teacher, and Student — each rendered based on the authenticated user's role. Key frontend features include a login/registration form with role selection, a class management interface for teachers, an assignment submission form for students, and a grading interface. All API requests are handled via fetch/axios calls to the Node.js backend, with JWT tokens stored in HTTP-only cookies for secure session management.

5.2 Backend

The backend was built using **Node.js** with the **Express.js** framework following a RESTful API architecture. It serves as the intermediary between the frontend and the MySQL database. Key API modules include: **Authentication** (registration, login, JWT token issuance), **User Management** (Admin CRUD operations on users, teacher validation), **Class Management** (class creation, enrollment via code), **Assignment Management** (create, view, submit, grade), and **Activity Logging** (recording user actions for audit purposes). Passwords are securely hashed using bcrypt, and role-based middleware restricts access to protected routes based on user roles (Admin, Teacher, Student).

5.3 Sequence Diagrams

Sequence diagrams illustrate how different components of the Student Tracking System interact with each other over time. These diagrams show the communication between users (Student, Teacher, Admin), the frontend (Next.js), the backend (Node.js API), and the database (MySQL).

The system follows a client-server architecture, where users send requests through the frontend interface. These requests are processed by the backend API, which interacts with the database to store or retrieve data. The backend then returns a response to the frontend, which updates the user interface accordingly.

The following sequence diagrams represent the key processes in the system:

- User Login
- Assignment Submission
- Assignment Creation
- Class Enrollment
- Assignment Grading
- User Management by Admin

These diagrams help to clearly visualize system workflows and ensure correct interaction between system components.

5.3.1 Login flow

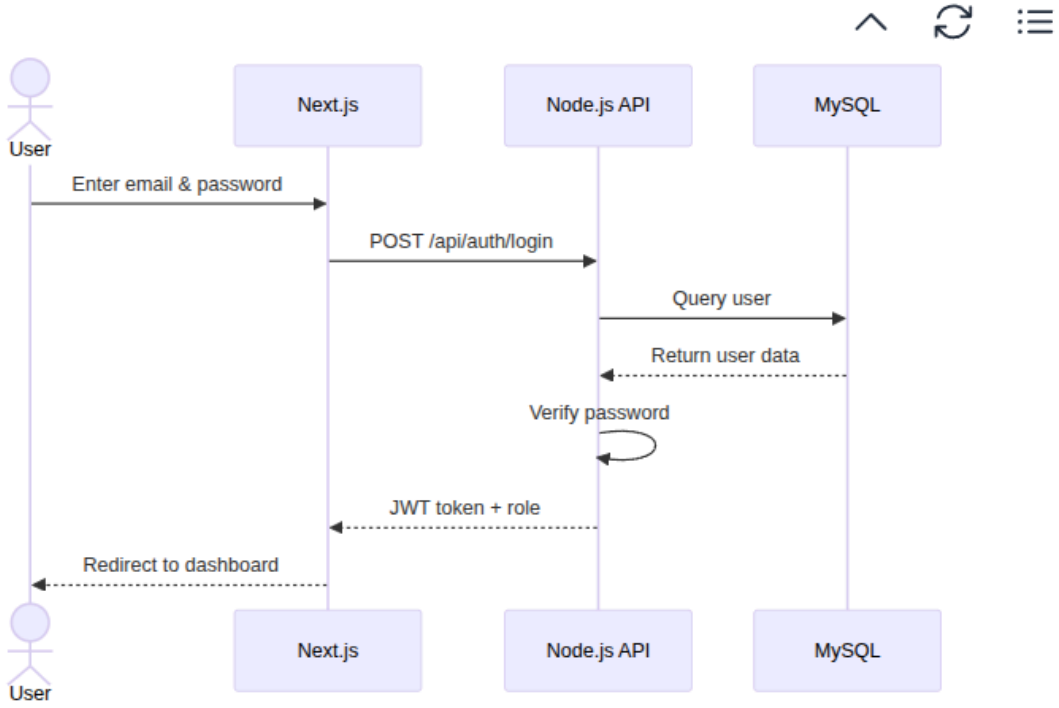


Figure X.X: Sequence Diagram for User Login

This diagram illustrates the authentication process. The user submits login credentials through the frontend, which sends a request to the backend API. The backend verifies the user information using the database and returns a JWT token upon successful authentication. The frontend then redirects the user to the appropriate dashboard.

5.3.2 Submit Assignment Flow

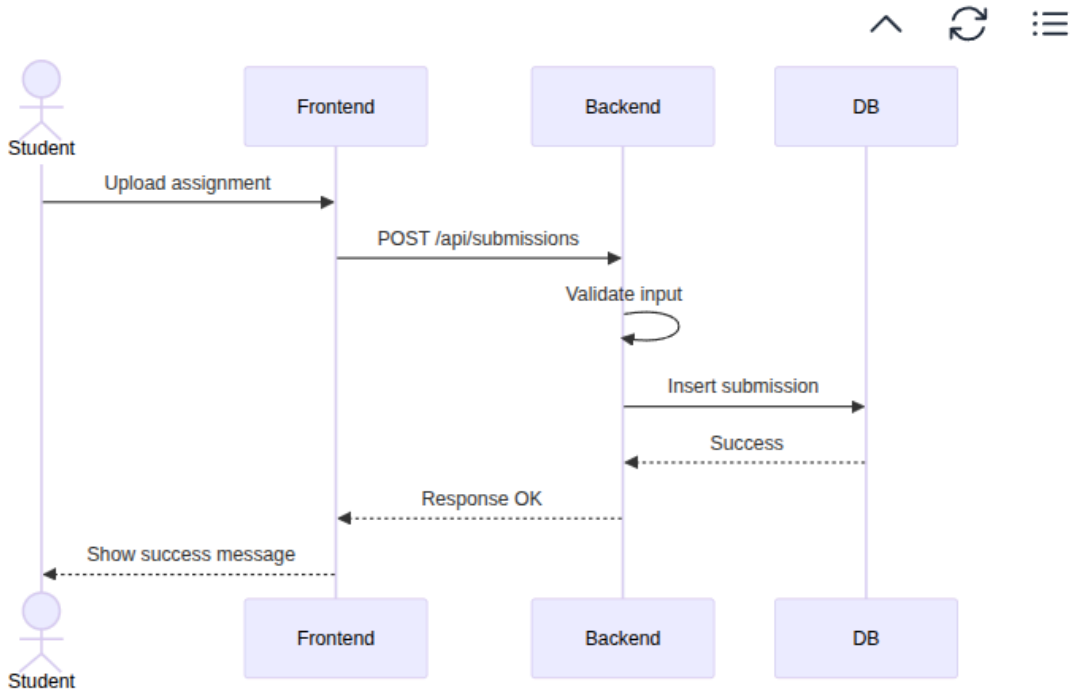


Figure X.X: Sequence Diagram for Assignment Submission

This diagram shows how a student submits an assignment. The frontend sends the submission data to the backend API, which validates and stores the data in the database. After successful storage, a response is returned to the frontend, and a confirmation message is displayed to the student.

5.3.3 Create Assignment Flow

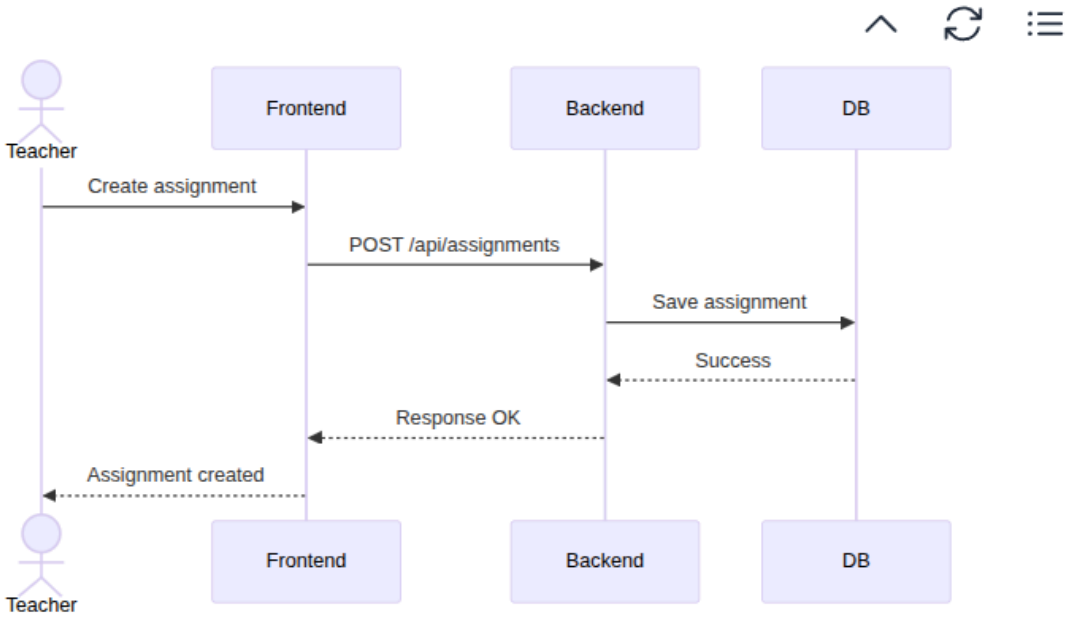


Figure X.X: Sequence Diagram for Assignment Creation

This diagram illustrates how teachers create assignments. The teacher inputs assignment details via the frontend, which sends a request to the backend API. The backend stores the assignment in the database and returns a success response to the frontend.

5.3.4 Join Class Flow

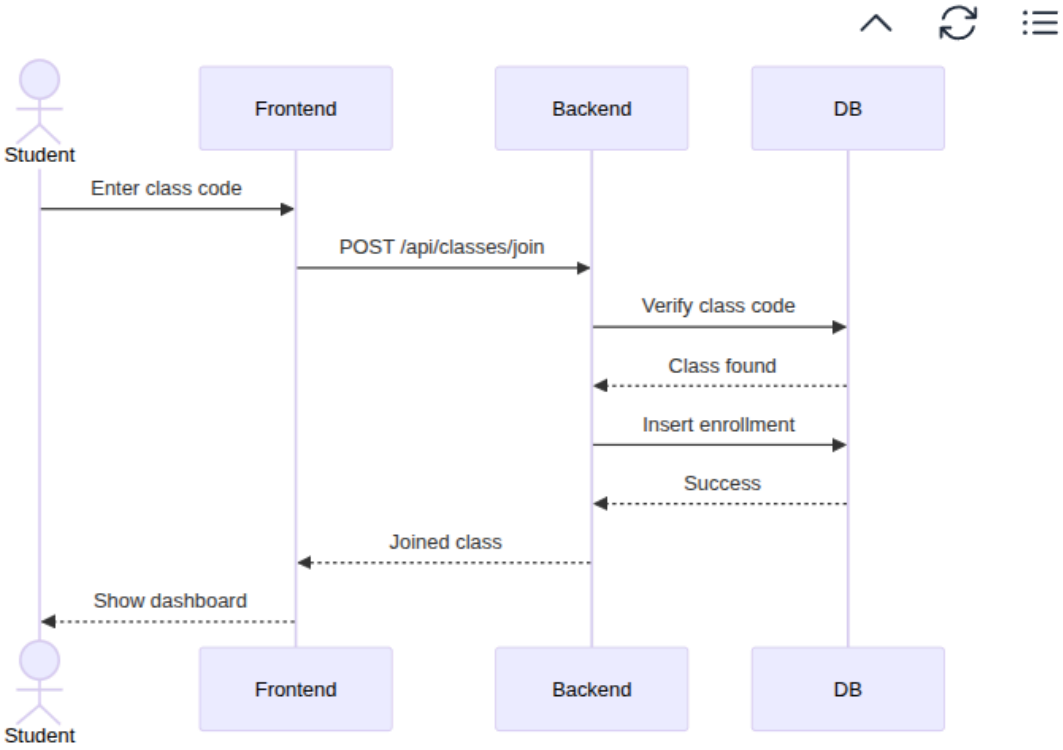


Figure X.X: Sequence Diagram for Class Enrollment

This diagram shows how a student joins a class using a class code. The system verifies the class code in the database and enrolls the student if valid. The frontend then updates to display the joined class information.

5.3.5 Grade Assignment Flow

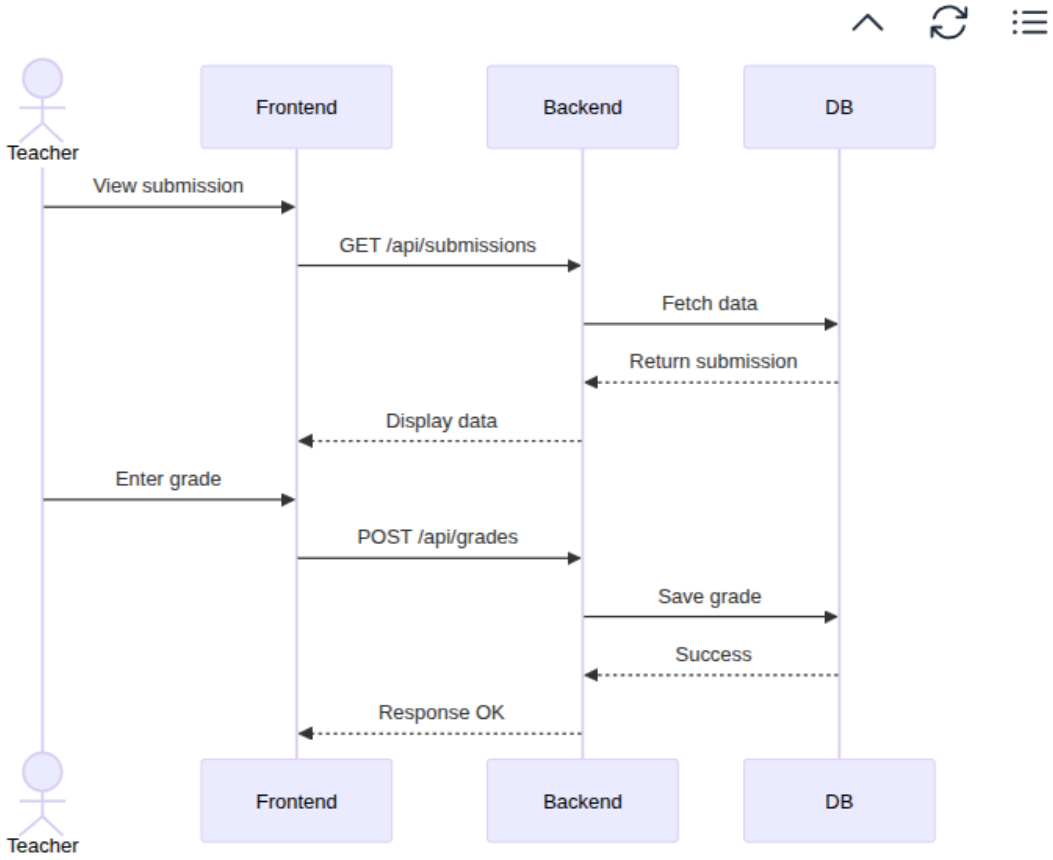


Figure X.X: Sequence Diagram for Assignment Grading

This diagram illustrates how teachers grade student assignments. The system retrieves submission data, allows the teacher to assign a grade, and stores the grading information in the database.

5.3.6 Admin Manage Users Flow

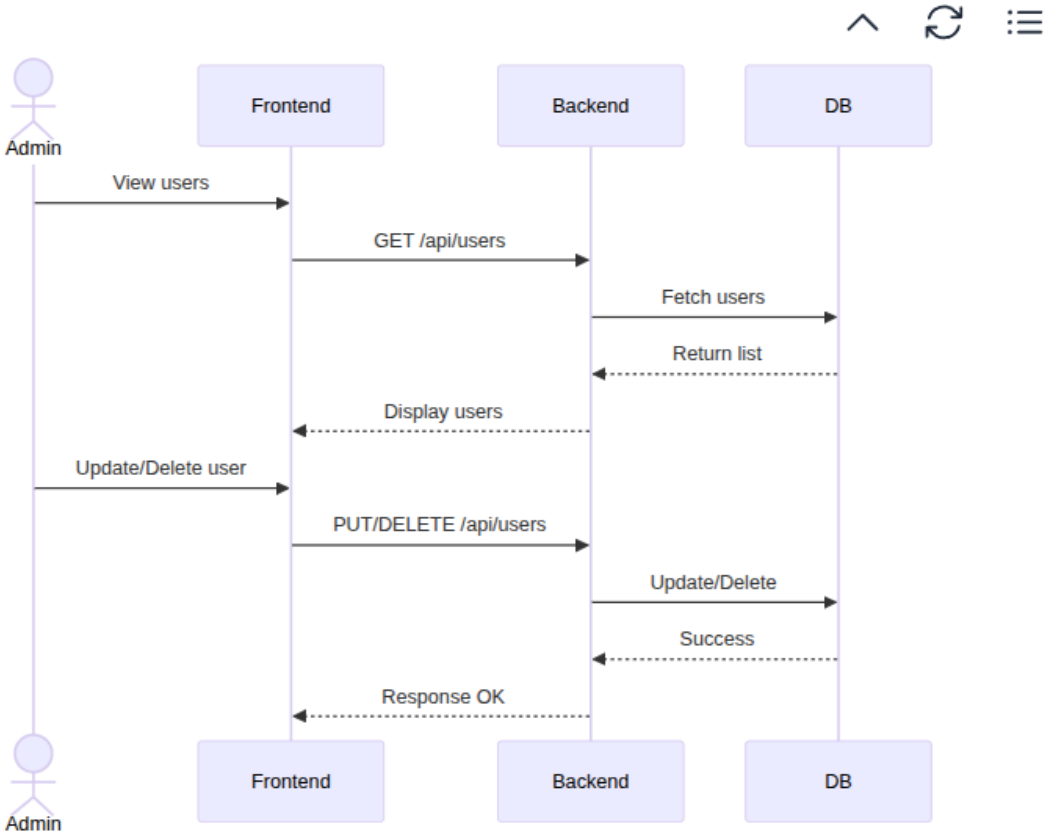


Figure X.X: Sequence Diagram for User Management

This diagram shows how the administrator manages users. The admin can view, update, or delete users through the frontend. The backend processes these requests and updates the database accordingly.

6. Testing

6.1. Functionality Testing

- All links work (no broken links).
- Forms validate inputs and show error messages.
- Buttons and navigation menus function correctly.
- Database operations (create, read, update, delete) work as expected.

b. **Usability Testing**

- The layout is intuitive and easy to navigate.
- Content is clear and readable in Khmer and English.
- Mobile responsiveness tested on multiple screen sizes.
- Accessibility features (alt text, keyboard navigation, ARIA labels).

c. **Interface Testing**

- API calls return correct data.
- Error handling is user-friendly (no raw error codes).
- Frontend and backend stay in sync (e.g., Firebase/RoomDB updates).

d. **Compatibility Testing**

- Tested on major browsers (Chrome, Firefox, Safari, Edge).
- Works on Android and iOS devices.
- Tested on different OS versions (Windows, macOS, Linux).

e. **Performance Testing**

- Page load time < 3 seconds.
- Stress test with high traffic simulation.
- Caching and CDN verified.
- Lighthouse audit for performance score.

f. **Security Testing**

- HTTPS enforced.
- Authentication and session management tested.
- SQL injection, XSS, CSRF vulnerabilities checked.
- Sensitive data encrypted.

g. **Additional Scenarios**

- Localization tested (Khmer/English, currency, date formats).
- Offline mode behavior checked.
- Push notifications tested (FCM delivery).
- Docker containers verified (health checks, database connectivity).
    - ❖ Suggested Tools
- **BrowserStack** : Cross-device / browser testing
- **Postman** : API testing
- **Lighthouse** : Performance & SEO audits
- **OWASP ZAP** : Security scanning
- **Jest / Cypress** : Automated frontend tests
- **Docker health checks** : Backend validation

7. Future Work

1. Integration with Mobile Application :

   Developing dedicated Android and iOS applications to provide administrators,

   teachers, and students with real-time access and improved communication on the go.

2. Advanced Reporting and Analytics :

Implementing advanced analytics and dashboards to track performance trends and

automate reporting, enabling management to make informed, data-backed decisions.

3. Integration with Other University Systems :

   Integrating with existing university infrastructure such as Learning Management

   Systems (LMS), libraries,and finance departments to create a unified academic platform.

4. Enhanced Security Features :

   Strengthening data protection by introducing two-factor authentication, end-to-end

   encryption, and comprehensive audit logs to safeguard sensitive information.

5. AI-Based Student Monitoring :

   Utilizing Artificial Intelligence to predict academic outcomes and identify "at risk" students, allowing for early intervention and personalized support.

8. Conclusion

The Student Tracking System project was successfully designed and developed to address the challenges of managing student information in educational institutions. The system provides a centralized, web-based platform that improves the efficiency, accuracy, and security of student data management compared to traditional manual methods.

Through the implementation of key features such as role-based access for administrators, teachers, and users, the system ensures that information is well-organized and securely managed. The user-friendly interface and responsive design also enhance usability, making it easier for users to interact with the system effectively.

Overall, this project demonstrates how information technology can support educational management by reducing manual workload, minimizing data errors, and improving accessibility. The Student Tracking System can serve as a strong foundation for future enhancements and wider adoption within educational institutions.

9. References

1. Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M.,
   Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C.,
   Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001).
   Manifesto for Agile Software Development.
   https://agilemanifesto.org/

2. Fedena. (2024). *Fedena Student Information System*. https://fedena.com
3. openSIS. (2024). *openSIS – Open Source Student Information System*. https://opensis.com
4. PowerSchool. (2024). *PowerSchool Student Information System*. https://www.powerschool.com
5. Figma, Inc. (2024). *Figma Design Platform*. https://www.figma.com
6. Draw.io. (2024). *Online Diagramming Tool*. https://www.drawio.com
7. W3Schools. (2024). *Web Development Tutorials*. https://www.w3schools.com
8. Next.js. (2024). *Next.js Documentation*. https://nextjs.org/docs
9. Tailwind Labs. (2024). *Tailwind CSS Documentation*. https://tailwindcss.com/docs
10. Node.js Foundation. (2024). *TypeScript Documentation*. https://www.typescriptlang.org/docs/
11. Docker Inc. (2024). *Docker Documentation*. https://docs.docker.com